
Projet Atlas

Script de démonstration du POC

La séquence exacte de commandes, dans l'ordre, pour présenter

Fil rouge de la démo :

1. Ça tourne → 2. C'est du code (IaC) → 3. C'est supervisé & résilient
→ 4. Les données sont protégées (RPO) → 5. On survit au désastre (RTO)

Durée cible : 12-15 min | **Objectifs prouvés :** IaC, sécurité, supervision, $RPO \leq 20$ min, $RTO \leq 40$ min

Table des matières

Avant de commencer	2
Aide-mémoire (1 page)	7

Avant de commencer (préparation, hors public)

Ouvrir à l'avance 4 onglets/terminaux pour enchaîner sans temps mort :

```
# Terminal A : sur le control node (hyperviseur), pour Ansible + pct
ssh root@138.201.135.108

# Terminaux B, C, D : les 3 tunnels (à laisser ouverts), sur son poste
ssh -L 8443:10.20.0.10:443 root@138.201.135.108 # GLPI
ssh -L 8080:10.30.0.12:8080 root@138.201.135.108 # Zabbix
ssh -L 3000:10.30.0.14:3000 root@138.201.135.108 # Grafana
```

Puis ouvrir 3 onglets navigateur : <https://localhost:8443>, <http://localhost:8080>, <http://localhost:3000>. Ne pas encore les montrer.

« Armer » l'incident chaos (indispensable)

Le trigger disque de Zabbix exige que le disque reste >90% **pendant 5 minutes** (anti-faux-positif) : l'alerte n'apparaît donc que **5 min après** le déclenchement. On **arme donc l'incident tout de suite**, avant l'Acte 1 ; il sera visible pile au moment de l'Acte 3.

```
# (Terminal A, TOUT AU DEBUT) declencher le remplissage disque sur la machine de lab
ssh root@138.201.135.108 'pct exec 208 -- /opt/atlas/scripts/chaos.sh disk'
```

Attention

Vérifier 5 min avant : les 3 tunnels montent, Zabbix affiche 10 hôtes verts (état sain de départ), et la machine de lab est propre (... `chaos.sh clean` si un incident cron traîne). **Puis** armer l'incident disque ci-dessus juste avant de commencer à parler.

ACTE 1 — « Ça tourne » : le service est vivant

(3 min)

1.1 — Toute l'infra est debout

```
# (Terminal A, sur le control node)
ssh root@138.201.135.108 'pct list'
```

🕒 À l'écran — ce que le public voit

Les **9 conteneurs LXC** sont tous en statut **running** (bastion, reverse proxy, GLPI, MariaDB, SMTP, Zabbix, backup, Grafana, testlab).

Ce qu'on dit

« Voici les 9 briques du POC, réparties sur 3 VLAN — DMZ, LAN-admin, management. Tout tourne. Regardons le service côté utilisateur. »

1.2 — Le portail GLPI répond (chemin utilisateur, en HTTPS)

```
# Onglet navigateur GLPI (tunnel déjà ouvert)
https://localhost:8443
```

◉ À l'écran — ce que le public voit

La page GLPI s'affiche en **HTTPS** (via le reverse proxy TLS). Se connecter, montrer la **liste des tickets** existants.

Ce qu'on dit

« Le trafic utilisateur entre par le reverse proxy chiffré, qui sert GLPI — la base n'est jamais exposée directement. »

1.3 — Tout est supervisé (Zabbix)

```
http://localhost:8080      # Monitoring -> Hosts
```

◉ À l'écran — ce que le public voit

10 hôtes en vert (Available) et *Monitoring* → *Problems* **vide**. C'est notre état de référence « tout va bien ».

1.4 — Les tableaux de bord (Grafana)

```
http://localhost:3000      # dashboard "Atlas - Vue d'ensemble"
```

◉ À l'écran — ce que le public voit

Grafana remonte les métriques (via un compte Zabbix en **lecture seule**). On voit CPU/RAM/disque des hôtes en direct.

ACTE 2 — « C'est du code » : Infrastructure as Code

(2 min)

2.1 — Rien n'est fait à la main : tout est reproductible

```
# (Terminal A) simulation : est-ce que le reel colle au code ?
ansible-playbook site.yaml --check --diff --tags bastion --limit atlbst01p
```

◉ À l'écran — ce que le public voit

Le playbook tourne en **mode simulation** et le bilan affiche **changed=0 failed=0** : l'infrastructure réelle est **déjà conforme au code**. Aucune dérive.

Attention

Utiliser le tag **bastion** (et non **zabbix**) : en mode **-check**, le rôle **zabbix** échoue faussement (une tâche crée le dossier TLS, simulé donc absent, et la tâche suivante ne le trouve pas). Le rôle **bastion** donne un **changed=0** propre.

Ce qu'on dit

« Toute l'infra est décrite en Ansible. Je peux la rejouer à l'identique : ici la simulation ne propose aucun changement, preuve que le serveur **EST** son code. C'est ce qui garantit qu'on saura tout reconstruire à l'identique en cas de sinistre. »

ACTE 3 — « C'est résilient » : chaos en direct

(3 min)

3.1 — Rappel : l'incident a été armé au début

On a lancé le remplissage disque en préparation (`chaos.sh disk` sur `atltst01p`) : le disque est monté à >90%. **5 minutes plus tard**, le trigger se déclenche — c'est maintenant.

Ce qu'on dit

« En début de démo, j'ai provoqué une panne réelle mais contrôlée sur la machine de lab : son disque s'est rempli. Voyons ce que la supervision en a fait, sans que j'aie touché à Zabbix. »

3.2 — Zabbix a détecté la panne tout seul

```
http://localhost:8080      # Monitoring -> Problems (rafraichir)
```

À l'écran — ce que le public voit

Un problème **rouge (Haute sévérité)** sur `atltst01p` : « **FS [/] : Space is critically low (used > 90%)** ». Détection automatique, aucune config en direct. (*Validé en répétition : alerte levée $\approx$ 5 min après le déclenchement.*)

3.3 — On répare, et l'alerte se résout

```
# (Terminal A)
ssh root@138.201.135.108 'pct exec 208 -- /opt/atlas/scripts/chaos.sh clean'
```

À l'écran — ce que le public voit

Le disque de `atltst01p` revient à $\sim$ 24%. Dans Zabbix, le problème repasse en **Resolved** et l'hôte redevient vert : la boucle « je casse $\rightarrow$ ça alerte $\rightarrow$ je répare » est bouclée.

Attention

`chaos.sh` a un garde-fou : il refuse de s'exécuter ailleurs que sur `atltst01p` (vérif `hostname -s`). Aucun risque pour les vrais services. **Ne pas utiliser le scénario service** (arrêt de `nginx`) : `nginx` n'est pas supervisé sur la machine de lab, donc il ne lèverait aucune alerte. Le scénario `disk` est le bon pour la démo.

ACTE 4 — « Les données sont protégées » : le RPO

(2 min)

4.1 — Sauvegarde toutes les 15 minutes

```
# (Terminal A) les derniers dumps sur la base
ssh root@10.30.0.10 'ls -lht /var/backups/atlas/db/ | head'
```

À l'écran — ce que le public voit

Une série de dumps espacés de **15 minutes** (`glpi_dump_AAAAMMJJ_HHMMSS.sql.gz`), le plus récent daté de moins de 15 min $\rightarrow$ **RPO $\leq$ 15 min** (mieux que la cible de 20 min).

4.2 — Répliqué hors-site

```
ssh root@10.30.0.13 'ls -lht /var/backups/atlas/db-remote/ | head'
```

À l'écran — ce que le public voit

Le même dernier dump est présent sur le serveur de sauvegarde **distinct** (atlbkp01p) : la perte de la base n'emporte pas les sauvegardes.

ACTE 5 — « On survit au désastre » : le PRA (RTO)

(3-4 min)

5.1 — Option courte à montrer en direct : restauration granulaire

```
# (Dans GLPI) supprimer un ticket de demonstration, puis :
# (Terminal A) choisir un dump anterieur
ssh root@10.30.0.10 'ls -lht /var/backups/atlas/db/ | head'

# Relancer uniquement les tickets manquants (sans ecraser les autres)
ansible-playbook pra_restore_granular.yaml \
  -e "pra_target_dump=/var/backups/atlas/db/glpi_dump_AAAAMMJJ_HHMMSS.sql.gz"
```

À l'écran — ce que le public voit

Le playbook restaure dans une base temporaire, réinjecte *seulement* les tickets absents (INSERT IGNORE), affiche le nombre de tickets, et journalise le RPO effectif. Rafraîchir GLPI : le **ticket supprimé est revenu**, les autres sont intacts.

Ce qu'on dit

« On peut réparer chirurgicalement une erreur humaine, sans toucher au reste. »

5.2 — Option complète (à décrire, ou à lancer si le temps le permet) : perte de VM

```
ansible-playbook pra_restore_full.yaml
```

À l'écran — ce que le public voit

Reconstruction de toute la chaîne dans l'ordre (MariaDB → GLPI → reverse proxy → supervision), restauration des données, vérifications automatiques (GLPI en HTTPS, comptage des tickets), et surtout : à la fin, le playbook **affiche le RTO mesuré** comparé à la cible de 40 min.

Ce qu'on dit

« En une seule commande, tout se reconstruit et se vérifie tout seul, et le temps de reprise est mesuré : c'est ça, tenir le RTO. »

Clôture — ce qu'on a prouvé

(1 min)

Exigence	Démontré par
100% IaC, reproductible	Acte 2 — simulation « 0 changed », infra = code
Sécurité (segmentation)	Acte 1 — HTTPS via reverse proxy, base non exposée, 3 VLAN
Supervision opérationnelle	Acte 3 — panne détectée puis résolue en direct
RPO $\leq$ 20 min	Acte 4 — dumps toutes les 15 min, répliqués hors-site
RTO $\leq$ 40 min	Acte 5 — restauration en une commande, RTO mesuré

Atlas — Script de démonstration du POC — Infrastructure as Code

Aide-mémoire — la démo en une page (à imprimer)

1. **Au tout début — armer le chaos** : `ssh root@138.201.135.108 'pct exec 208 - /opt/atlas/scripts/disk'` (l'alerte arrive $\approx$ 5 min après, pile pour l'Acte 3).
2. **Ça tourne** : `ssh root@138.201.135.108 'pct list'` → 9 running ; puis les 3 onglets : GLPI (HTTPS), Zabbix (10 verts), Grafana.
3. **C'est du code** : `ansible-playbook site.yaml -check -diff -tags bastion -limit atlbst01p` → `changed=0`.
4. **Chaos** : Zabbix → Problems affiche « FS [/] : Space is critically low » (rouge) → réparer : `ssh ... 'pct exec 208 - .../chaos.sh clean'` → résolu.
5. **RPO** : `ssh root@10.30.0.10 'ls -lht /var/backups/atlas/db/ | head'` + `ssh root@10.30.0.13 '... db-remote ...'`.
6. **RTO** : `ansible-playbook pra_restore_granular.yaml -e "pra_target_dump=..."` (ou `pra_restore_full.yaml`).